

Final Technical Report: Contributions and Justifications

Félicien Moquet

Contents

1	Introduction	2
2	User Documentation Service	2
3	Privacy Policy Integration	2
4	Email Sending Service: Design & Robustness	2
5	Client-Side Validation and UX	3
6	Diagrams	3
7	First Deployment Strategy: GitHub Pages	4
8	Possible Improvements	5
9	Personal Evaluation	6

1. Introduction

In this project, my contributions focused on delivering robust, usable, and legally responsible features. These efforts included the integration of documentation services, real-time email functionality, client-side validation mechanisms, and streamlined public deployment.

The aim was to ensure both technical operability and a clear user experience while respecting the legal obligations of AI monitoring tools.

2. User Documentation Service

A comprehensive user manual was created and made available in PDF format. This documentation provides first-time users with detailed instructions, complete with screenshots, explaining how to navigate the AI-powered proctoring system.

It includes:

- Webcam activation
- Email registration
- Session result interpretation
- Common troubleshooting tips

Link: [User Documentation PDF](#)

3. Privacy Policy Integration

A privacy policy was developed to clarify the system's scope and ethical use. It outlines how data is (or isn't) stored, the probabilistic nature of AI, and warns that results should not be used for punitive actions without human oversight.

Users must explicitly accept this policy via a checkbox before beginning any session. This ensures GDPR-style compliance and informed consent.

Link: [Privacy Policy PDF](#)

4. Email Sending Service: Design & Robustness

A real-time notification feature was implemented using **EmailJS**, a client-side email dispatching API. It enables the system to send session summaries to both candidate and sponsor immediately after the monitoring session ends.

Justification for EmailJS

- **No backend required:** It runs fully in-browser, avoiding the need for Node.js, server APIs, or SMTP configuration.
- **Quick integration:** EmailJS has native JavaScript support, which fit well with our Vite/React stack.

- **Asynchronous:** The API uses Promises, which simplifies error handling and feedback via `async/await`.

Limitations

- The free tier limits to 200 emails/month.
- Email addresses can be spoofed unless tied to a verified sender.
- EmailJS does not guarantee GDPR or HIPAA compliance.

Error Handling & Validation

The `sendEmail()` function was written to include:

- **Strict input validation:** All three fields (results, candidate email, sponsor email) are checked.
- **Regex-based email format validation.**
- **Error propagation:** Wrapped in `try/catch` to prevent UI crash and log detailed feedback.
- **Alerts:** Visual feedback is immediately provided to users via small red error messages or alerts.

5. Client-Side Validation and UX

To ensure the user enters valid and complete information, multiple layers of validation were added in the `ButtonHandler.jsx` component:

- **Required fields:** Empty inputs are blocked with an error message.
- **Real-time checkbox monitoring:** The "Terms of Use" box must be checked.
- **Conditional rendering:** Errors only appear after attempting to start the session, keeping the form clean.
- **Visual feedback:** Invalid or missing input displays red error messages in smaller font, styled with CSS.

Error Visibility Logic

Errors are only displayed once the user clicks "Start Session". This avoids overwhelming the user on initial load, while still enforcing strict validation.

6. Diagrams

To better illustrate the system's architecture and workflow on the user interface, I created the following diagrams:

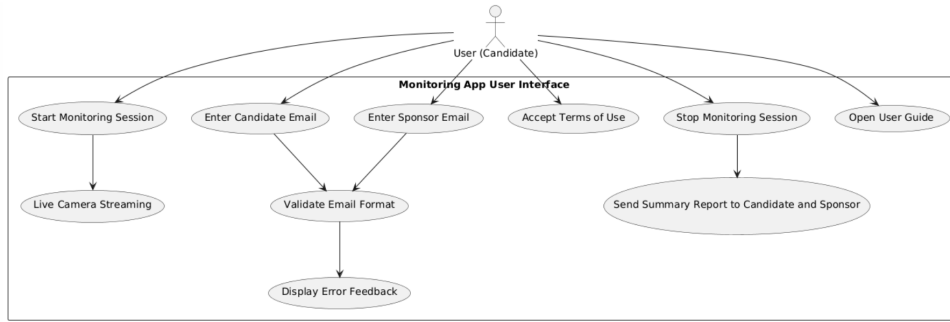


Figure 1: User interface use case diagram

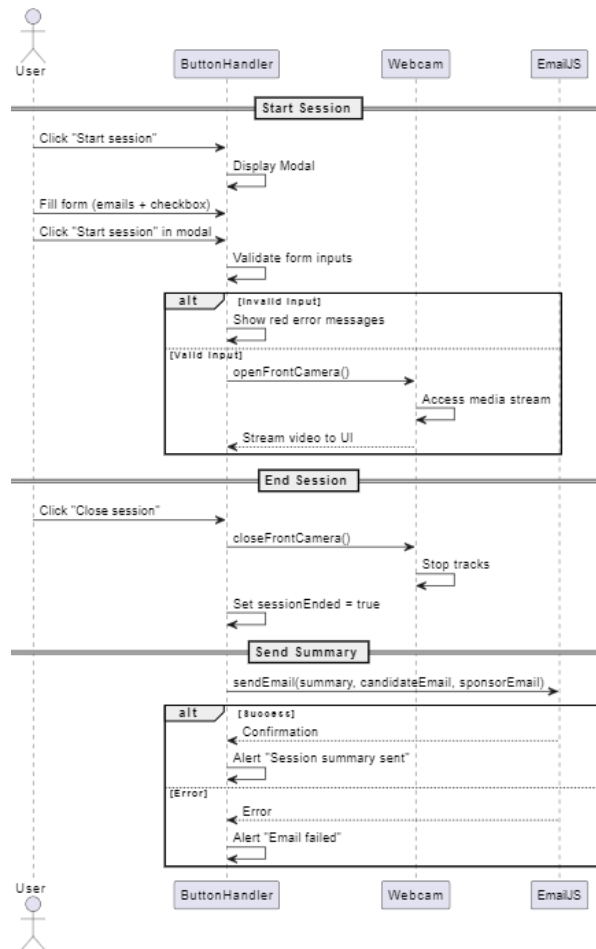


Figure 2: User Interface Workflow Diagram: This diagram illustrates the user flow, from starting a session to receiving email summaries.

7. First Deployment Strategy: GitHub Pages

Initially, the project was deployed publicly using GitHub Pages (as we had only a frontend app at this time), leveraging the `/docs` directory in the repository. This offered a lightweight, quick way to demo the application and validate user-facing functionality without infrastructure costs.

Additionally, since the project was already hosted and version-controlled on GitHub, using GitHub Pages significantly simplified deployment during the testing phase. It required no external setup, allowing us to iterate rapidly and publish updates directly from the development branch.

Deployment Process

- `vite.config.js` was configured to set `base: "."` and output the build to `docs/`.
- PDF files such as the user manual and privacy policy were manually copied post-build.
- A custom `yarn deploy` script automates the build, commit, and push process.

Problems Encountered

- **PDF overwrite issues:** The Vite build occasionally deleted non-code assets unless copied after the build step.
- **Invisible file changes:** Git would not detect modified binaries like PDFs unless explicitly forced.
- **CI/CD errors:** GitHub Actions failed due to phantom submodule links in `node_modules/.cache`

Migration to AWS by Kateryna

After validating the GitHub Pages deployment, **Kateryna redeployed the system with a full backend on AWS Cloud infrastructure.**

The initial GitHub Pages version was kept as a test/demo environment, while the AWS deployment serves as the foundation for a more robust production-ready architecture.

8. Possible Improvements

- **Custom domain:** To remove GitHub branding and offer a clean URL.
- **Server-based email service:** Replace EmailJS with a backend like NodeMailer + Gmail API or SendGrid.
- **Dedicated email address:** Create a system-specific email address for sending notifications, as the current implementation uses my personal email with EmailJS (on github Pages) or Kateryna's one (on AWS).
- **Security:** Add CSP headers, reCAPTCHA, and token-based user validation.
- **AI Improvements:** Current spoof detection model could be improved with post-processing or ensemble models.
- **Monitoring dashboard:** Add an admin UI to track sessions, logs, and detection stats (now allowed thanks to AWS and the backend system).

9. Personal Evaluation

This project allowed me to develop skills at the crossroads of UX design, frontend engineering, ethical AI deployment, and devops automation.

My focus was on making the system:

- Intuitive to use (via clear UI and documentation)
- Legally responsible (via policy agreements)
- Technically robust (with validation, exception handling, and fallback alerts)